

THE CLAUDE MANUAL

Level One

From running tasks to building things.

Now with diagrams, worked examples and step-by-step procedures.

Prompting · Projects · Skills · Connectors · Data & GDPR · Source & Git · Code · Agents

The sequel to Level Zero — for when the basics feel easy.

By Hans Tandt

Ardennes, Wallonia, Belgium

Version 2.1 — August 2026

How to use this book

This is Level One. It assumes you have done the things in Level Zero: you can set up Claude, run a task in Cowork, find your files, and manage a session. If those are not yet comfortable, read Level Zero first — this book builds straight on it.

Level One is about the jump from running one-off tasks to building things that keep working: better prompting, reusable skills, connected tools, data you can trust, shared source code, a gentle step into Code, and your first small automated agent.

Three paths — read what you need

New to all of this: read in order.

You have the basics and want output quality: chapters 1–5 (prompting, projects, skills, connectors, data & GDPR).

You need teamwork now: jump to chapters 7–9 (where source lives, Git, working as a team) — they stand on their own.

This book is a snapshot — Claude changes fast

Everything here reflects Claude as of August 2026: the model families Opus, Sonnet, Haiku and Fable, the Cowork desktop app, and the unified home that shares Chat, Cowork and your Projects in one sidebar. Anthropic ships updates very fast — often in days, not months — so a screen, a menu name or a model number may already have moved. When what you see does not match the page, trust the app and claude.com. The ideas and habits here hold even when the buttons move.

What is new in version 2.1

Added: "Record a skill" in Cowork (Chapter 3) and Claude Design (Chapter 10).

Updated: where Projects now live in the new single app home, and a reminder that models and screens change quickly.

Table of Contents

A one-page recap of Level Zero

Before we build, the five ideas everything here rests on:

1. **Pick the right face.** Chat to think, Cowork to produce files, Code to build software, Projects to remember, Connectors to reach other apps.
2. **Context beats cleverness.** What Claude can see before it answers matters more than how you phrase the prompt.
3. **Let Claude ask first, then verify.** "Ask me questions first"; and never trust a confident answer without checking it.
4. **A file is real only in your folder.** Until it is saved there, it lives in the sandbox or scratchpad and can vanish.
5. **Keep sessions small.** Start fresh per topic; lean on session notes and memory so you never re-explain yourself.

1. Prompting that actually works

Level Zero said context beats cleverness. Level One makes that practical. A good request has four parts, and none of them is a magic word.

A good request has four parts (not a magic word)



Give the destination and the standards — not a long list of steps.

Part	What it means	Example
Goal	The outcome you want, in one sentence.	"Draft a one-page offer letter."
Context	The facts, files, and constraints Claude needs.	"Use the attached template and our standard terms."
Examples	One or two samples of good (and bad) output.	"Match the tone of this past letter; avoid this phrasing."
Quality bar	How you will judge it; what 'done' looks like.	"Formal, under 250 words, no buzzwords."

1.1 Context in action — before / after

Same task, two prompts. **Before:** "Write an email to a supplier about the late order." You get a generic, bland email. **After:** "Write an email to our supplier Dupont about order 4471, 9 days late. Firm but polite, ask for a new delivery date and a partial credit, under 120 words, sign as me." You get something you can almost send. The difference is not cleverness — it is context.

1.2 The anti-style file

The fastest way to stop AI-sounding output is to write down what you never want. A short "anti-style" file — words you never use, openings you hate, patterns you reject — does more than any positive instruction. Keep it in your ABOUT ME folder so every session reads it.

1.3 Show, then correct

Don't aim to one-shot important work. Ask for a draft, react to it, and let Claude adjust. Three quick rounds of "good, but shorter and less formal" beat thirty minutes crafting the perfect first prompt.

1.4 Verify the answer — what, and how

Claude sounds as sure when wrong as when right. Keep Web Search on, and ask it to audit itself: "Check your own answer before I trust it; verify the facts and tell me what you are unsure about." For anything that matters, check against the source too.

2. Projects in depth

A Project is a folder of chats that share instructions, files, and memory. Here is how to make one that saves time instead of becoming a junk drawer. Since mid-2026 your projects live in the left sidebar of the unified home, shared between Chat and Cowork — pin the ones you use so they are easy to find.

2.1 One project per deliverable

The common mistake is a single giant project for "everything." The fix is one project per recurring deliverable — a weekly report, a type of proposal, a specific tool. Each gets only the files it needs and instructions written for that one job.

2.2 What goes in the instructions

Project instructions are not documentation — they are the minimum that stops Claude drifting. Five to eight lines:

- What this deliverable is, and who it is for.
- The tone and the quality bar.
- What to always do, and what to never do.
- Where to save the result, and in what format.

Files are not free

More files do not mean better answers — they mean more noise to filter. Upload examples of good work and the templates you reuse, not your whole archive. If an answer drifts, the cause is often too many files, not too few.

2.3 Procedure — create a useful project in 5 minutes

1. Name it after one deliverable (e.g. "Weekly ops report").
2. Add only that deliverable's templates and 1–2 good past examples.
3. Write 5–8 lines of instructions (what / who / tone / where to save / when to ask).
4. Test with one real task; have Claude self-critique and tighten the instructions.

3. Building your first Skill

A Skill is your standard way of doing one job, written so Claude runs it the same way every time. If you have ever written a checklist, you can build a skill — no code required.

3.1 What a skill really is

A skill is a small text file named `SKILL.md`. It has a short description at the top and the full instructions below. Only the description sits in Claude's memory; the rest loads when a task matches. That is why you can have many skills without slowing anything down.

3.2 Procedure — create and test a skill

1. **Do the task once by hand** with Claude, paying attention to the steps and the mistakes.
2. **Harvest it:** "Turn what we just did into a reusable skill. Write the steps as clear instructions, add a short 'gotchas' section, and keep the description to one line so it triggers on the right requests."
3. **Test on a fresh example** that is slightly different, and fix whatever the skill missed.
4. **Save it** in the app's Settings, under Capabilities. A skill only improves when its file is edited — it does not learn on its own.

The fastest way to build one: record yourself doing it

Since mid-2026 Cowork has a "Record a skill" option in the + menu of the desktop app. Instead of writing the steps, you record your screen and talk through the task once as you do it, and Claude turns the demonstration into the skill. It is the easiest route for a job that is simpler to show than to describe — especially browser tasks that have no connector. Do not record passwords or private data. On Pro, Max and Team.

4. Connectors — letting Claude reach your other tools

A connector is a secure bridge between Claude and another app — email, calendar, a drive, a project tracker. With one connected, Claude can read and act on data that lives outside your folder.

4.1 Connect only what the task needs

The instinct is to connect everything. Resist it. Each connector adds noise and uses context. A morning-briefing task needs your mail and calendar — not your whole toolchain. Turn connectors off when a workflow does not use them.

4.2 The three kinds of connector

Connectors run on a shared standard called **MCP** (Model Context Protocol). You browse and add them from the Connectors settings (the "+" inside a chat, or Settings, Connectors). There are three kinds, and the difference matters for trust:

- **Anthropic's built-in integrations** — first-party (Gmail, Drive, GitHub, Slack...).
- **Directory connectors** — third-party, but reviewed and vetted by Anthropic before listing.
- **Custom MCP connectors you add yourself** — nobody has reviewed these; treat them with the same caution as any new app you install.

4.3 A first safe use

Start read-only. Ask Claude to summarize today's unread mail, or list this week's meetings, before you let it send or change anything. Build trust on tasks where a mistake is harmless, then widen from there.

5. Data, security and GDPR before you connect

The moment Claude touches real business data — through a connector, a pasted document, or a folder — data protection becomes your responsibility. This chapter is practical awareness, not legal advice; when in doubt, ask your data-protection officer.

5.1 The pre-connection data checklist

1. What data will Claude see, and does any of it identify a person?
2. Do I have a reason (a lawful basis) to process it this way?
3. Is it the minimum needed for the task, or am I over-sharing?
4. Could I anonymise or redact it and still get the result?

5.2 GDPR in plain language

The EU's GDPR governs personal data — anything that identifies a living person (names, emails, staff or customer records, an accident file). A few principles cover most day-to-day cases:

- **Have a reason.** Process personal data only for a clear, legitimate purpose.
- **Use the minimum.** Don't feed in more personal detail than the task needs.
- **Keep it purposeful.** Don't reuse data for something people wouldn't expect.
- **Protect and delete.** Keep it secure, and don't keep it longer than needed.

Where does the data go? (and can I keep it local?)

Chat, Cowork and Code all send your content to Anthropic's servers to be processed — Claude does not run on your computer, and no setting makes your data stay local. You CAN turn off using your conversations to train the models (Settings, Privacy), which also shortens retention. Team, Enterprise and API accounts do not train on your data by default and offer stronger retention controls. For truly local, offline AI you would need a different, self-hosted model — which is not Claude.

5.3 Trace and keep control

For anything sensitive, keep a light record of what you fed in and why, prefer the traffic-light rule from Level Zero (green / orange / red), and make sure a human reviews before anything is sent or acted on. If a connector can act (send mail, change records), start it read-only until you trust it.

Not legal advice

This chapter is general awareness to help you avoid obvious mistakes. GDPR obligations depend on your role and your organisation — confirm specifics with your data-protection officer before processing personal data at scale.

6. Running projects in parallel

You are not limited to one thing at a time. You can write a document in Cowork while Claude fixes an app in Code, and keep several Cowork tasks moving at once. The tools run independently — the limit is you, and your quota.

6.1 What can run at the same time

- **Cowork and Code together.** Two separate windows, two separate jobs, no interference.
- **Several sessions.** Different chats or projects, each with its own context, side by side.
- **Background and scheduled tasks.** Work that ticks along without you watching, and even Dispatch tasks from your phone.

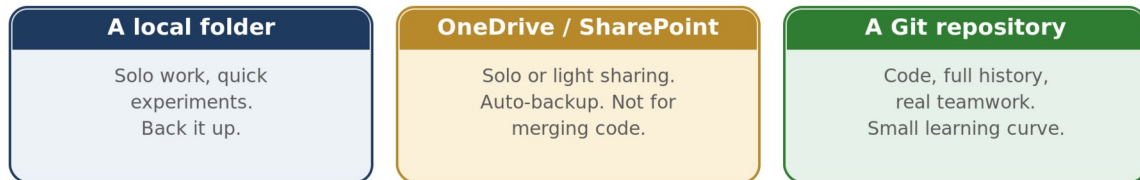
6.2 The two real costs

1. **Your attention.** The machines don't get confused, but you do. Switching between a document and a bug every minute is tiring and error-prone. Batch by context: an hour on writing, then an hour on the app.
2. **Your quota.** Two heavy jobs draw on the same 5-hour and weekly limits. Running them together spends your allowance faster, not slower.

7. Where your project source lives

Once a project matters — or more than one person touches it — where the files live stops being an afterthought.

Where your project source should live



Alone on docs → local + OneDrive. Code, or a team → Git.

7.1 A simple rule of thumb

- **Alone on documents?** A local folder backed up by OneDrive is fine.
- **Alone on code?** Use Git anyway — a full history and an undo button for the whole project.
- **With others on the same files?** Git for code; a shared SharePoint or Drive library for documents.

7.2 Keep one tidy structure

Whatever you choose, point Cowork or Code at a clean project folder with an obvious shape: source files, a place for outputs, a notes or README file, and nothing unrelated. A tidy folder makes Claude faster and your teammates calmer.

8. A gentle introduction to Git

Git gives a project a memory and a safety net. It is the single most useful thing to learn when you move from solo experiments to real, shared work — and far less scary than it looks.

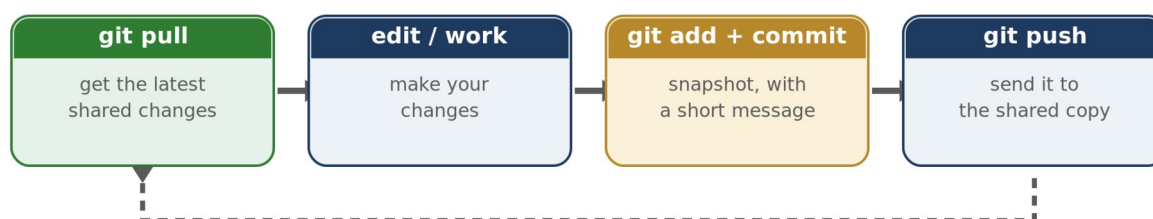
8.1 What Git actually does

Git takes snapshots of your project whenever you ask. Each snapshot (a "commit") is a labelled point you can return to. So you can change anything fearlessly: if it breaks, you go back to the last good snapshot. Nothing is ever truly lost.

Word	In plain terms
Repository (repo)	Your project folder, plus its full history of snapshots.
Commit	One saved snapshot, with a short message saying what changed.
Push / Pull	Send your commits to the shared copy online / get others' commits down.
Branch	A safe side-track to try something without touching the main version.
Clone	Make your own local copy of a shared repo.

8.2 The everyday loop

The everyday Git loop — four steps, repeated



You can just tell Claude in Code: "commit my changes with a sensible message and push."

GitHub (or GitLab, or — in a Microsoft shop — Azure DevOps) is the website that hosts the shared copy. You can ask Claude in Code to run the steps for you — "commit my changes with a sensible message and push" — but knowing what they mean still helps.

8.3 Workshop — your first repo in 10 minutes

1. In Code, open a project folder and say "set this folder up as a Git repository."
2. Make one small change, then "commit it with a clear message."
3. Create a free GitHub account; say "create a private repo and push this to it."
4. Change something again, commit, push — and watch the history build up online.

Two rules that prevent most Git pain

- 1) Pull before you start and commit often in small pieces — tiny snapshots are easy to understand and undo.
- 2) Never put secrets (API keys, passwords) in a repo. Once pushed, assume they are public. Keep them in a separate local file that Git is told to ignore.

9. Working as a team on the same project

The moment two people touch the same files, you need a way to combine their work without overwriting each other.

9.1 Documents vs code

- **Shared documents** (Word, Excel, slides): a shared SharePoint or Drive library tracks versions and shows who changed what. Avoid two people editing the exact same file at the same second.
- **Shared code** (scripts, apps, anything text-based): use Git. It is built precisely to merge changes from several people safely.

9.2 The team Git loop

1. **Pull first** so you build on what others did.
2. **Work on a branch** named for the task, not on the main version.
3. **Open a pull request** — a teammate glances at it, then it joins the main version.

That review step is not bureaucracy — it is a second pair of eyes, and where shared understanding lives. When two people change the same lines, Git asks which to keep (a "merge conflict"); it sounds alarming but is routine, and you can ask Claude in Code to explain and resolve it.

10. A gentle first step into Code

Claude Code writes and runs real software. You do not have to read or write the code yourself — you describe what you want in plain language and Claude does it.

10.1 Describe, don't type

"Add a button that exports the table to Excel" is a complete instruction. Claude proposes the change, you approve it, and it runs. You stay the decision-maker; Claude does the typing.

10.2 CLAUDE.md — the project's brain

Every Code project should have a file called `CLAUDE.md` at its root. It is read at the start of every session and holds how the project is laid out, how to run it, the choices you made, your preferences. After a good first session, say "write a `CLAUDE.md` capturing everything you have learned about this project" and keep it updated.

10.3 Permissions — three levels, not two

- **Ask every time** (default): safe, slow.
- **Auto-accept edits** but confirm risky commands: a good middle ground.
- **Bypass all permissions** (fast "vibe" mode): only in a dedicated, Git-backed folder whose files you could replace — never on your home directory.

10.4 Plan first, then build

For anything non-trivial, ask Claude to investigate and propose a short plan before writing a line — "show me two options and wait for my choice." Approving a plan up front prevents long detours and keeps you in control.

10.5 Vibe coding — and its limits

What you are doing here has a name: vibe coding — building software by describing what you want and steering by feel, instead of reading and writing the code. It shines for personal tools, prototypes and throwaway scripts. For anything other people depend on, the rules tighten: keep it in Git, actually run and test what it produces, and never ship what you cannot check. Vibe code freely on low-stakes things; slow down and verify on anything that matters.

10.6 Claude Design — interfaces without code

A cousin of Code, launched in 2026 as a research preview at claude.ai/design, Claude Design builds the look of things — mockups, landing pages, simple websites and slide decks — from a plain description. Unlike an image generator it produces the real thing: a working layout with buttons you can actually click, not a picture of one. Reach for it when what you want is a page or a design rather than software. It is on Pro, Max, Team and Enterprise; being a research preview, expect it to keep changing.

11. Your first small agent

An agent is just Claude working toward a goal across several steps, often on a schedule, and reporting back. You don't need a grand system — one useful, repeating task is a real agent.

11.1 What makes it an agent

Three ingredients: a clear job, a trigger (a time or an event), and a way to tell you the result. Everything else is detail.

11.2 A complete case — a morning briefing

1. **Define the job:** "Every weekday at 7am, list my meetings today and any unread mail flagged important, and write a 5-line summary."
2. **Make it a saved task** so it runs the same way each time, with the mail + calendar connectors read-only.
3. **Schedule it** for 07:00 on weekdays (your machine, or an always-on one, must be awake).
4. **Send it somewhere you'll see it** — an email to yourself, or a message on your phone.

Start with a dry run

Run it by hand a few times and read the output critically before you schedule it. An agent repeating a small mistake every morning is worse than no agent.

11.3 Grow it slowly

Add one capability at a time, and keep a short notes file of what it does and how to fix it. Just because Claude can add another feature in seconds does not mean it should exist.

12. Tools and apps to install

You can go far with just the desktop app. But a handful of free, standard tools make building smoother. Install them as you need them — not all at once.

Tool	What it is for	When you need it
A code editor (VS Code, or Notepad++)	Reading and lightly editing text and code files.	As soon as you open a project's files yourself.
Git + a GitHub account	Version history and teamwork (Chapters 8–9).	The moment a project matters or is shared.
Python and Node.js	The engines most small tools and scripts run on.	When Code builds something you want to run locally.
PuTTY / WinSCP	Connecting to and copying files to a server or Pi.	When you run tasks on an always-on machine.
A messaging bot (e.g. Telegram)	Letting an agent send you alerts and reports.	When you want results pushed to your phone.

Let Claude set them up

Say "walk me through installing Git on Windows, one step at a time, and check each step with me." The same works for Python, Node, or connecting to a Pi.

13. Staying maintainable

The difference between a fun weekend and a system you still trust in six months is a few quiet habits.

- **Back up in two places.** Source in Git, important folders synced or zipped. One copy is not a backup.
- **Commit and note as you go.** Small commits and a short README mean future-you understands a project quickly.
- **Use memory and session notes at scale.** A MEMORY.md and per-project notes keep every new session oriented.
- **Prune skills and connectors** you stopped using — dead ones are clutter and risk.
- **Know when to stop.** Before adding a feature, ask whether it earns its keep. Subtraction is a skill.

The one-sentence test

For any project, you should be able to say in one sentence what it does and why it exists. If you cannot, it has grown without direction — prune it before you extend it.

Glossary of new terms

Term	In plain words
Repository (repo)	A project folder plus its full history of saved snapshots, managed by Git.
Commit	One saved snapshot, with a short message describing the change.
Push / Pull	Send your commits to the shared online copy / bring others' down to yours.
Branch	A safe side-track to try changes without touching the main version.
Merge conflict	When two people changed the same lines; Git asks which to keep. Routine.
Pull request	A request to merge your branch into the main version, usually reviewed first.
GitHub / GitLab	Websites that host the shared copy of a Git repo so a team can share it.
CLAUDE.md	A file at a Code project's root telling Claude how the project works.
MCP	The shared standard connectors use to link Claude to other apps.
GDPR	The EU rules for handling personal data — anything that identifies a living person.
Agent	Claude working toward a goal across several steps, often scheduled, reporting back.

What's next

If you have worked through Level Zero and this Level One book, you can now set Claude up well, produce real deliverables, handle data responsibly, keep your source safe and shared, take a first step into Code and Claude Design, and run a small agent. That is most of what most people ever need.

The API — the step beyond this book

You may have heard of the **Claude API** (part of the Claude Platform). The app, Cowork and Code all let you use Claude through a window someone else built. The API lets a program of your own talk to Claude directly — billed per token — so you can build your own apps and agents. It is more technical, which is why it sits beyond this book; but it is closer than it sounds — the moment a small tool you make in Code calls Claude on its own, it is already using the API.

And keep the two habits from Level Zero close: **let Claude ask first, and never trust a confident answer without checking it.** They matter more, not less, as what you build gets bigger.

By Hans Tandt